SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I S Deepak_1921260077 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Errors in Equivalence Partitioning and Boundary Value Analysis (Online Shopping)

10 Marks

1. Error Identification

Assumed flawed design. The *Quantity* field of the shopping cart accepts whole numbers from **1 to 10** (business rule). The tester produced only two classes and two test cases:

Tester's class	Range used	Test value	Expected result
Class A	0 to 10	5	Accepted
Class B	above 10	15	Rejected

Table 1: The flawed test design

#	Error found	Why it is wrong
E1	Valid and invalid values mixed in Class A (0 to 10)	0 is invalid, 1 to 10 is valid; one representative cannot stand for both behaviours.
E2	Negative numbers missing	Values below 0 are a separate invalid class that must be rejected.
E3	Non-numeric, blank and decimal inputs ignored	Input-type classes (abc, empty, 2.5) are part of the input domain.
E4	No boundary values	Defects cluster at 0/1 and 10/11, yet none of these were tried.
E5	Class B treated as one class	Very large values (e.g. 99999) can trigger overflow or stock-check failures.

Table 2: Errors in partitioning and boundary analysis

2. Root Cause Analysis

- **Misunderstanding of the equivalence principle:** a class must contain values that the program treats identically. The tester grouped by *numeric look* (0 to 10) instead of *behaviour* (accept or reject).
- **Specification read only once:** the lower limit 1 was overlooked, so 0 slipped into the valid class.
- **Only the valid domain was modelled:** invalid classes were derived from a single 'greater than 10' idea, ignoring type and sign.
- **BVA treated as optional:** boundaries were assumed to be covered by 'nearby' values, but off-by-one defects (`<` instead of `<=`) live exactly on the edges.

3. Corrected Test Design

The input domain is split into one valid class and several invalid classes, and BVA adds the values on and around each edge.

Class	Description	Type	Representative
EC1	Quantity < 0	Invalid	-5
EC2	Quantity = 0	Invalid	0
EC3	1 <= Quantity <= 10	Valid	5
EC4	Quantity > 10	Invalid	50

Class	Description	Type	Representative
EC5	Non-numeric text	Invalid	abc
EC6	Blank input	Invalid	(empty)
EC7	Decimal number	Invalid	2.5

Table 3: Corrected equivalence classes

Boundary	Values tested	Expected outcome
Lower	0, 1, 2	Reject, Accept, Accept
Upper	9, 10, 11	Accept, Accept, Reject

Table 4: Boundary value test cases

Final suite

Seven equivalence-class tests plus six boundary tests (-5, 0, 1, 2, 5, 9, 10, 11, 50, abc, blank, 2.5) give about 12 distinct test cases that cover every class and every edge.

4. Justification

- **Every class is covered:** each invalid class has its own test, so rejection and error-message logic is checked separately.
- **Boundary defects are exposed:** a coding error such as `qty > 1` or `qty < 10` fails at the boundary tests 1 or 10 immediately.
- **Efficiency:** 12 targeted tests replace an unbounded set of random values and still detect the most probable faults.

5. Conclusion

The original design failed because of impure classes and missing boundaries. Separating valid from invalid input and adding BVA yields a small, systematic suite with much higher defect detection effectiveness.

Q2. Evaluation of Random Testing in a Banking Application

10 Marks

1. Error Identification

Assumed context. A funds-transfer rule: minimum **Rs. 100**, maximum **Rs. 50,000** per transaction, daily limit **Rs. 1,00,000**. The QA engineer ran about 200 random transfers (e.g. 2,300; 17,850; 9,420; 33,000) and reported 'high coverage'. Critical defects at 99/100 and 50,000/50,001 were missed.

#	Error in strategy or assumption	Consequence
E1	Number of tests equated with coverage	200 random values may all fall into the same one or two partitions.
E2	No requirement-based coverage measure	Nobody can show which rules were tested.
E3	Boundaries never selected	Random chance of hitting exactly 100 or 50,000 is under 0.1%.
E4	Invalid inputs not planned	Negative, zero, non-numeric amounts were never tried.
E5	Combination rules (daily limit, balance) ignored	Limit-breach and insufficient-balance defects survive.
E6	Results not repeatable	Failures cannot be reproduced or regression tested.

Table 1: Errors in the random testing approach

2. Root Cause Analysis

- **Random sampling is uniform, defects are not.** Faults sit on edges such as `amount <= max` versus `amount < max`; uniform sampling almost never reaches these single points.
- **No test model.** Without partitions or a decision model there is no definition of 'enough', so volume was used as a proxy for adequacy.
- **Coverage assumption confusion:** 'many tests executed' was mistaken for 'many behaviours exercised'.
- **No traceability** from requirements (limits, balance, daily cap) to test cases.

3. Corrected Approach: Black Box Techniques

Technique	Application to the transfer module	Sample test values (Rs.)
Equivalence Partitioning	Classes: < 100 (invalid), 100 to 50,000 (valid), > 50,000 (invalid), non-numeric	50, 5,000, 70,000, abc
Boundary Value Analysis	Edges of both limits	99, 100, 101, 49,999, 50,000, 50,001
Decision Table	Amount valid? x Balance sufficient? x Daily limit available?	8 rules, one test each
State Transition	Session states: Logged-in, OTP-pending, Locked	3 wrong OTPs lock the account

Table 2: Techniques selected

Amount valid	Balance enough	Daily limit left	Expected action
Y	Y	Y	Transfer succeeds
Y	Y	N	Reject: daily limit exceeded

Amount valid	Balance enough	Daily limit left	Expected action
Y	N	Y	Reject: insufficient funds
Y	N	N	Reject: insufficient funds
N	-	-	Reject: invalid amount

Table 3: Reduced decision table (invalid amount collapses four rules)

4. Justification of Adequacy and Reliability

- **Test adequacy becomes measurable:** 100% of partitions, 100% of boundary points and 100% of decision rules.
- **Reliability:** every business rule has at least one failing-capable test, so a defect in any rule produces a visible failure.
- **Repeatability:** each case has fixed input and expected output, enabling regression automation.
- **Random testing is retained only as a supplement** (for example fuzzing) after structured tests pass.

Evaluation verdict

Random testing alone is unreliable for limit-based logic. A structured black box suite of roughly 20 cases finds the missed defects that 200 random tests could not.

5. Conclusion

Coverage must be defined against the specification, not against test count. EP, BVA, decision tables and state testing provide that definition and close the gaps behind the missed transaction-limit defects.

Q3. Incomplete Control Flow Graph and Path Testing — 10 Marks

1. Error Identification

Assumed module. The code reads `x` and `n`, selects a mode with `if (x > 0)`, loops `while (i < n)` over an array counting even elements, and finally prints HIGH if `sum > 5` else LOW.

#	Error in developer's CFG / path testing	Effect
E1	Decision node `x > 0` drawn as a plain statement	Only one of two branches is ever tested.
E2	`while` loop and its back edge missing	Zero-iteration, one-iteration and multi-iteration behaviour untested.
E3	Decision `a[i] even ?` omitted	Odd and even element paths never separated.
E4	Decision `sum > 5` omitted	HIGH output path never reached.
E5	Only one path derived	Path coverage far below the achievable 5 independent paths.

Table 1: Errors in the control flow graph

The incomplete graph has N = 6 nodes and E = 5 edges, giving V(G) = 5 - 6 + 2 = 1: it describes a straight-line program, whereas the real module has four decisions.

2. Root Cause Analysis

- **CFG built from the algorithm outline, not from the code.** Decisions hidden inside loop bodies and compound statements were skipped.
- **Loops not modelled as cycles.** A loop needs a condition node and a back edge; drawing it as a block removes the iteration dimension.
- **Cyclomatic complexity never computed,** so there was no check on how many independent paths were expected.
- **Coverage driven by test count,** not by path enumeration.

3. Corrected Control Flow Representation

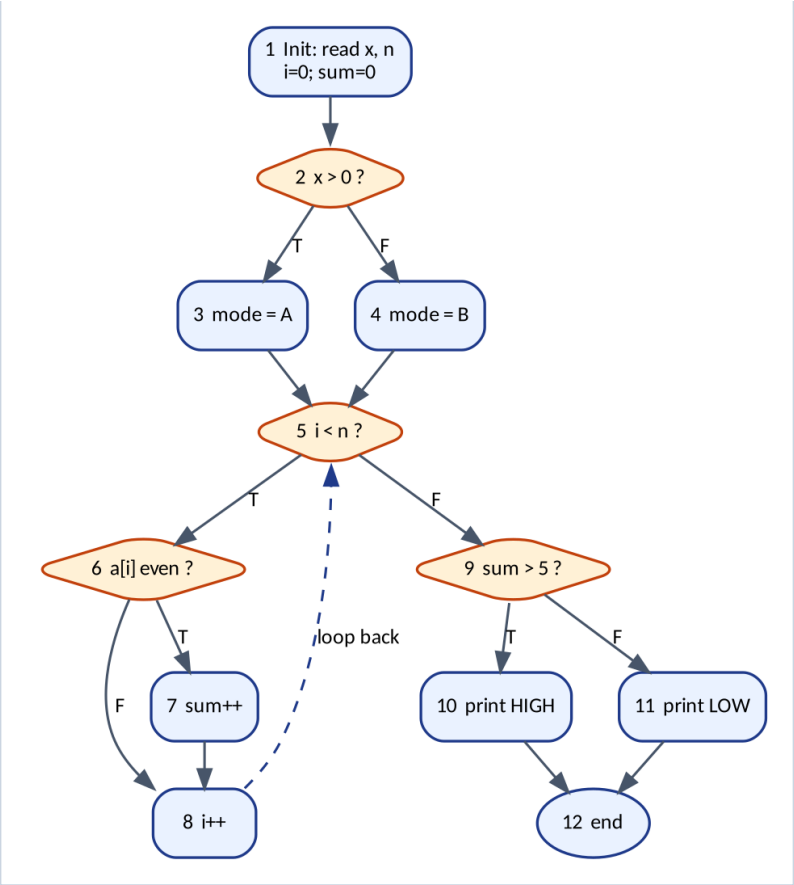


Figure 1: Corrected CFG (4 decision nodes, 1 loop)

Cyclomatic complexity. Nodes N = 12, edges E = 15, so **V(G) = E - N + 2 = 15 - 12 + 2 = 5**. Check: predicate nodes (2, 5, 6, 9) + 1 = 5.

Path	Node sequence	Test data	Expected
P1	1-2-3-5-9-11-12	x=5, n=0	LOW (loop skipped)
P2	1-2-4-5-9-11-12	x=-5, n=0	LOW, mode B
P3	1-2-3-5-6-7-8-5-9-11-12	x=5, a=[2]	LOW, sum=1
P4	1-2-3-5-6-8-5-9-11-12	x=5, a=[3]	LOW, odd element
P5	1-2-3-5-(6-7-8-5) x 6-9-10-12	x=5, a=[2,2,2,2,2]	HIGH, sum=6

Table 2: Independent paths (basis set)

4. Justification

- The basis set of 5 paths exercises **every edge at least once**, so statement and branch coverage reach 100%.
- Loop entry, loop skip and loop repetition are all present (P1, P3, P5), exposing initialisation and off-by-one faults.
- V(G) = 5 gives a **mathematical upper bound on tests needed**: fewer than 5 means incomplete coverage.
- Each path has a defined expected result, so a failing path points directly at the faulty decision.

5. Conclusion

A correct CFG with all decisions and the loop back edge exposes five independent paths instead of one. Testing these paths provides measurable path coverage and far better fault detection.

Q4. Why 100% Statement Coverage Missed Logical Errors

10 Marks

1. Error Identification

Assumed module (loan grading):

```
1  grade = 'B';
2  if (score > 700 || income > 50000)   // spec: score >= 700
3      grade = 'A';
4  return grade;
```

The tester ran **one test**: `score = 800, income = 20000`. Lines 1 to 4 all executed, so the report said 100% statement coverage. Yet the module contains a boundary defect (`>` instead of `>=`) and the false branch of the decision was never executed.

Gap	Evidence
Decision never evaluated to False	Only the True outcome was tried, so branch coverage is 1 of 2 = 50%.
Individual conditions not isolated	`income > 50000` never decided the outcome; its correctness is unknown.
Boundary defect hidden	score = 700 was not tested; the program returns B but A is expected.
No oracle for the false path	'Nothing happens' paths have no statement to execute.

Table 1: Gaps in coverage criteria

2. Why Statement Coverage Alone Was Insufficient

- Statement coverage only asks: *was each line executed?* An `if` with no `else` has an implicit false path that contains **no statement**, so skipping it costs nothing.
- It ignores **how the condition was evaluated**. Compound conditions with `||` or `&&` can hide wrong operators or constants.
- It cannot detect **missing statements** (missing else logic).
- It is the weakest white box criterion; it subsumes nothing else.

3. Recommended Improved Strategies

Criterion	What it demands	Tests for the module
Statement	Each statement once	1 test
Branch (Decision)	Decision True and False	2 tests
Condition	Each atomic condition True and False	2 tests
MC/DC	Each condition independently flips the outcome	N + 1 = 3 tests
Path	Each independent path (V(G) = 2)	2 tests

Table 2: Hierarchy of white box coverage

Test	score	income	Expected	Actual	Purpose
T1	800	20000	A	A	Decision True via score
T2	600	20000	B	B	Decision False (both False)

Test	score	income	Expected	Actual	Purpose
T3	600	60000	A	A	Income alone makes True (MC/DC for income)
T4	700	20000	A	**B**	Boundary: exposes the `>` defect

Table 3: Revised test cases

> **Result**
> T2 and T3 reach 100% branch coverage and MC/DC; T4 reveals the defect that 100% statement coverage concealed.

4. Justification

- **Branch coverage** forces both outcomes of every decision, uncovering missing else behaviour.
- **Condition / MC/DC** proves each operand matters, catching wrong operators and constants.
- **Path coverage** guards combinations across several decisions and loops.
- Cost increases only modestly (2 to 4 tests here) while fault detection rises sharply.

5. Conclusion

Statement coverage is a necessary minimum, not proof of correctness. Combining branch, condition (MC/DC) and path coverage gives a stronger adequacy criterion and detects logical errors in conditionals.

Q5. Equivalence Partitioning for an Age Field (0 to 100) — 10 Marks

1. Error Identification

The tester defined only two classes: **Negative numbers** and **Positive numbers**.

#	Error	Explanation
E1	Zero belongs to neither class	0 is neither negative nor positive, yet it is a valid age (lower boundary).
E2	'Positive' mixes valid and invalid values	Ages 1 to 100 are valid; 101 and above are invalid but sit in the same class.
E3	Valid vs invalid limits not modelled	The specification range 0 to 100 is never reflected in the classes.
E4	No Boundary Value Analysis	-1, 0, 1, 99, 100, 101 are never tested.
E5	Invalid data types missing	Letters, symbols, blank and decimals (25.5) are untested.

Table 1: Errors in the partitioning strategy

2. Root Cause Analysis

- Classes were based on the **sign of the number**, a mathematical property, instead of the **validation rule** of the field.
- The *valid* range was not treated as a class bounded by two edges; hence no lower and upper limit emerged.
- The tester assumed that *any positive value behaves like any other*, which is false once an upper limit (100) exists.
- BVA was absent, so the typical off-by-one faults (`age < 100` vs `age <= 100`) were invisible.

3. Corrected Partitions

Class	Range / Condition	Validity	Test value
EC1	age < 0	Invalid	-10
EC2	0 <= age <= 100	Valid	45
EC3	age > 100	Invalid	150
EC4	Non-numeric (letters, symbols)	Invalid	abc, @#
EC5	Blank	Invalid	(empty)
EC6	Decimal value	Invalid	25.5

Table 2: Corrected equivalence classes

Boundary	Test values	Expected
Lower edge	-1 / 0 / 1	Reject / Accept / Accept
Upper edge	99 / 100 / 101	Accept / Accept / Reject

Table 3: Boundary Value Analysis (six values)

Complete suite

-10, -1, 0, 1, 45, 99, 100, 101, 150, abc, blank, 25.5 : twelve test cases cover all classes and all boundaries.

4. Justification

- **Valid versus invalid limits are explicit,** so both the accept path and every reject path are exercised.
- **Robust input validation:** type, range and presence checks are tested independently, preventing crashes and bad data from entering the database.
- **Boundary testing** catches the highest-probability faults with just six values.
- **Minimal yet complete:** one representative per class avoids redundant testing.

5. Conclusion

Replacing the sign-based split with range-based classes and adding boundary analysis converts a weak two-case test into a systematic suite that validates the 0 to 100 rule completely.

Q6. Condition Coverage Claim for a Compound Decision

10 Marks

Assumption

The question text is cut off after `if(A && B`. It is assumed here that the statement is `if (A && B || C)`, a compound decision with three atomic conditions A, B and C. The same reasoning applies to any similar expression.

1. Error Identification

The developer used the test pair **T1: (A,B,C) = (T,T,F)** and **T2: (A,B,C) = (F,F,T)** and claimed *full condition coverage*.

Test	A	B	C	A && B	Decision (A&&B)||C
T1	T	T	F	T	**T**
T2	F	F	T	F	**T**

Table 1: The developer's tests

- Every atomic condition took both True and False, so *condition coverage* is technically 100%.
- But the **decision was True in both tests**. Decision (branch) coverage is only 50%: the else branch was never executed.
- No test shows that each condition **independently** changes the outcome, so wrong operators (for example `A || C`) survive.

2. Root Cause Analysis

- **Condition coverage looks at operands, not at the decision outcome.** It does not subsume decision coverage.
- **Masking:** with `||`, a True C hides the values of A && B; with `&&`, a False A hides B (short-circuit evaluation may never evaluate B).
- The developer chose the **minimum number of tests (2)** to flip every operand and never checked outcomes.

3. Correction: Coverage Criteria Compared

Criterion	Requirement	Minimum tests	Weakness
Condition	Each condition T and F	2	Decision may not flip
Decision	Decision T and F	2	Ignores individual conditions
Condition/Decision	Both of the above	2 to 3	Masking still possible
MC/DC	Each condition independently affects outcome	N + 1 = 4	Strongest practical criterion

Table 2: Comparing criteria

Row	A	B	C	Decision
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	F

Row	A	B	C	Decision
5	F	T	T	T
6	F	T	F	F
7	F	F	T	T
8	F	F	F	F

Table 3: Full truth table

Condition	Independence pair (rows)	What changes
A	2 and 6	A flips T/F, B=T and C=F fixed, outcome T to F
B	2 and 4	B flips, A=T and C=F fixed, outcome T to F
C	4 and 3	C flips, A=T and B=F fixed, outcome F to T

Table 4: MC/DC pairs

> **Minimal MC/DC set**
> Rows 2, 3, 4 and 6: (T,T,F)=T, (T,F,T)=T, (T,F,F)=F, (F,T,F)=F. Four tests give 100% condition, decision, condition/decision and MC/DC coverage.

4. Justification

Take the faulty variant `A || C` (operator error). The developer's tests T1 and T2 both return True for it, so the fault is **not detected**. In the MC/DC set, row 4 (T,F,F) gives True instead of the expected False, so the fault is exposed. Four tests rather than eight give near-exhaustive fault detection (N + 1 instead of 2^N).

5. Conclusion

Full condition coverage does not guarantee decision coverage. MC/DC adds only two tests yet proves the independent influence of every condition, which makes it the right criterion for complex boolean logic.

Q7. Decision Table Testing for a Discount System

10 Marks

1. Error Identification

Business rules (assumed). Premium members get 10%; a valid coupon gives 15%; when both apply the discounts add up but are **capped at 20%**. An expired coupon gives no discount and displays an error message.

#	Error in the QA team's decision table	Effect
E1	Only three rules built: Premium only, Coupon only, Neither	The Premium + Coupon combination is absent.
E2	Coupon validity (valid / expired) not a condition	Expired-coupon behaviour untested.
E3	No action for the 20% cap	Discounts stack to 25% or compound to 23.5%.
E4	Rules not derived from 2^n combinations	No completeness check.
E5	Expected prices not stated	No test oracle for price calculation.

Table 1: Errors in the table construction

2. Why the Current Approach Is Incomplete

- Conditions were considered in isolation, but the defect lives in their **interaction**.
- Three rules for two conditions: 2^2 = 4 rules were required, so one is plainly missing. With validity added, 2^3 = 8 combinations exist (6 after merging don't-care rules).
- The developer applied the discounts sequentially (1000 x 0.90 x 0.85 = 765, a 23.5% reduction) instead of capping at 20% (800). That arithmetic error can only be found by the missing combination rule.

3. Revised, Exhaustive Decision Table

Conditions: C1 = Premium member, C2 = Coupon applied, C3 = Coupon valid (don't-care when no coupon). Order value for the examples is Rs. 1000.

Rule	C1 Premium	C2 Coupon	C3 Valid	Action: discount	Price (Rs.)
R1	Y	Y	Y	20% (10+15 capped)	800
R2	Y	Y	N	10% + 'coupon expired'	900
R3	Y	N	-	10%	900
R4	N	Y	Y	15%	850
R5	N	Y	N	0% + 'coupon expired'	1000
R6	N	N	-	0%	1000

Table 2: Complete decision table (8 raw combinations reduced to 6 rules)

Test	Rule covered	Input	Expected
TC1	R1	Premium, valid coupon, 1000	800
TC2	R2	Premium, expired coupon, 1000	900 and error message
TC3	R3	Premium, no coupon, 1000	900
TC4	R4	Non-premium, valid coupon, 1000	850
TC5	R5	Non-premium, expired coupon, 1000	1000 and error message

Test	Rule covered	Input	Expected
TC6	R6	Non-premium, no coupon, 1000	1000

Table 3: One test case per rule

4. Justification of Effectiveness

- **Completeness:** all 2^3 = 8 combinations are accounted for, with merged don't-care cases clearly documented.
- **Interaction defects detected:** TC1 fails on any implementation that stacks or compounds discounts beyond the cap.
- **Negative scenarios covered:** TC2 and TC5 check that invalid coupons never reduce the price.
- **Low cost:** six tests protect the production price engine; each rule is traceable to a requirement.

5. Conclusion

The original table missed the interaction rules that produce price errors. The revised exhaustive table makes every Premium and Coupon combination explicit, and testing each rule prevents calculation defects from reaching production.

Q8. Loop Testing: Simple and Nested Loops

10 Marks

1. Error Identification

Assumed code: `for (i = 1; i <= max; i++) { process(a[i]); }` with `max = 50`. The tester ran only n = 1 and n = 50.

#	Error in the strategy	Defect that escapes
E1	Zero-iteration case (n = 0) skipped	Uninitialised result, null pointer, division by zero on empty input
E2	n = max + 1 never tried	Off-by-one: array index out of bounds, buffer overrun
E3	n = 2, typical and max - 1 untested	Faults needing a second pass (state carried between iterations)
E4	Nested loops not considered	Interaction between inner and outer counters
E5	No expected result per iteration count	Wrong totals pass unnoticed

Table 1: Errors in the loop test strategy

2. Root Cause Analysis

- **Assumption that extremes suffice.** The tester treated n = 1 and n = max as the only boundaries and forgot that the loop has *two* boundaries: entry (n = 0) and exit (n = max + 1).
- **Loop modelled as a block, not as a decision.** The loop condition `i <= max` is a decision with a False edge at entry; testing it requires skipping the body entirely.
- **Reliance on the 'happy path'.** Boundary-crossing inputs (n = 0, max + 1) look invalid and were thought irrelevant.
- **No standard loop-testing checklist** (Beizer) was applied.

3. Corrected Approach

(a) Simple loop (maximum n = 50):

Test	Iterations	Purpose	Expected
L1	0	Skip the loop	Empty result / default handled
L2	1	Minimum pass	One element processed
L3	2	Second pass, carried state	Two elements processed
L4	25 (m)	Typical value	Normal result
L5	49 (max - 1)	Just below the limit	49 processed
L6	50 (max)	Exact limit	50 processed
L7	51 (max + 1)	Overrun attempt	Rejected or error, no overflow

Table 2: Simple loop test cases

(b) Nested loops (outer O up to 10, inner I up to 20). Beizer's method avoids the explosion of 7 x 7 = 49 combinations:

1. Start with the **innermost loop**; set the outer loop to its minimum (1 iteration).

2. Run the simple-loop tests on the inner loop (0, 1, 2, typical, max - 1, max, max + 1) while keeping outer iterations at the minimum.
3. Move outward: hold the inner loop at a typical value and apply the simple-loop tests to the outer loop.
4. Add out-of-range values for each loop; continue until all loops are tested.
5. For **concatenated loops** that share a counter, treat them as nested; if independent, test separately.

Step	Outer count	Inner count	Aim
N1	1	0, 1, 2, 10, 19, 20, 21	Inner boundaries
N2	0, 1, 2, 5, 9, 10, 11	10	Outer boundaries
N3	10	20	Maximum load (200 iterations)

Table 3: Nested loop test plan (about 17 tests instead of 49+)

4. Justification of Path Integrity

- Cases L1 and L7 exercise both the **True and False edges** of the loop decision at entry and exit, so branch coverage of the loop is complete.
- Intermediate counts (2, typical) prove the loop variable updates correctly, preserving **path integrity** across iterations.
- The nested strategy limits tests to a *linear* number while still reaching boundary combinations.
- Defects at n = 0 and n = max + 1, the ones missed originally, are now detected directly.

5. Conclusion

Loop faults occur at the boundaries of iteration counts. Testing 0, 1, 2, typical, max - 1, max and max + 1 on a simple loop, and building outward from the innermost loop for nested ones, gives complete and economical loop coverage.

Q9. Error Guessing versus Structured Testing in a Safety-Critical Braking System

10 Marks

1. Error Identification

#	Error in relying solely on Error Guessing	Why it is dangerous
E1	Test selection depends on tester's memory and intuition	Different testers find different faults; results are not repeatable.
E2	No systematic coverage of input ranges	Pedal force, wheel slip and speed limits are never verified at the edges.
E3	State behaviour untested	Transitions such as ABS to Fault are never checked.
E4	No measurable adequacy criterion	Management cannot judge whether testing is 'enough'.
E5	No traceability to safety requirements	Certification standards (ISO 26262) require documented, structured test design.
E6	Unknown unknowns missed	Only anticipated faults are targeted.

Table 1: Errors in the ad-hoc approach

2. Impact on Test Adequacy

- **Coverage is unknown and uneven:** experienced intuition clusters on familiar faults, leaving whole partitions unchecked.
- **Residual risk is unquantified,** which is unacceptable where a defect can cause injury.
- **Regression testing is unreliable** because ad-hoc tests are not recorded as exact input and expected output.

3. Proposed Strategy: BVA + State Transition Testing

Boundary Value Analysis. Assumed requirements: pedal travel 0 to 100%, vehicle speed 0 to 250 km/h, ABS triggers when wheel slip exceeds 20%.

Input	Valid range	Boundary tests	Expected
Pedal travel (%)	0 to 100	-1, 0, 1, 99, 100, 101	Reject, idle, brake, ... , full brake, clamp / reject
Speed (km/h)	0 to 250	-1, 0, 1, 249, 250, 251	Sensor error at -1 and 251; valid otherwise
Wheel slip (%)	trigger at > 20	19, 20, 21	No ABS, no ABS, ABS active

Table 2: BVA test values

State Transition Testing. The braking controller has four states.

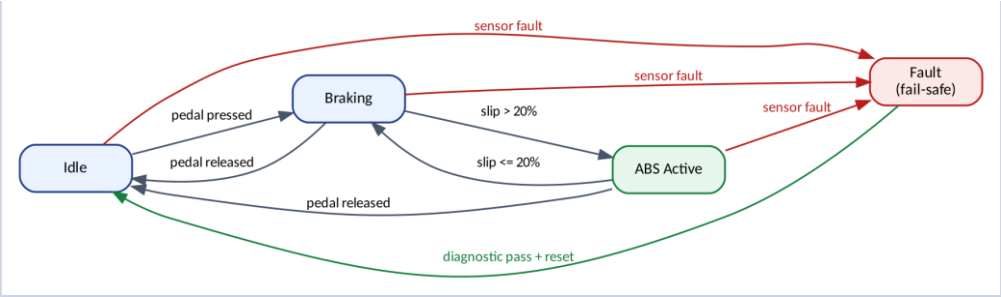


Figure 1: State transition diagram of the braking controller

State	Pedal pressed	Pedal released	Slip > 20%	Slip <= 20%	Sensor fault
Idle	Braking	Idle (ignore)	Idle (ignore)	Idle (ignore)	Fault
Braking	Braking	Idle	ABS Active	Braking	Fault
ABS Active	ABS Active	Idle	ABS Active	Braking	Fault
Fault	Fault (mechanical backup)	Fault	Fault	Fault	Fault

Table 3: State table including invalid events (negative tests)

4. Justification of Increased Safety Assurance

- **0-switch coverage:** every valid transition (9) and every invalid (state, event) pair in Table 3 is tested at least once.
- **Fail-safe check:** transitions into Fault from every state prove the system never remains in an unsafe state.
- **BVA** catches threshold faults such as `slip >= 20` instead of `slip > 20`, which directly changes when ABS engages.
- **Error guessing is retained as a complement** (e.g. sensor noise, sudden power dip) but runs after the structured suite, targeting what the models cannot predict.
- Results are **traceable, repeatable and measurable**, matching safety-standard expectations.

5. Conclusion

For a braking system, intuition-only testing leaves untested states and unmeasured boundaries. Combining BVA with state transition testing, and keeping error guessing as a supplement, delivers documented, quantifiable safety assurance.

Q10. Missing Else Branches in a Control Flow Graph

10 Marks

1. Error Identification

Assumed code (nested validation with implicit else branches):

```
if (userValid) {                 // decision 1
    if (amount > 0) {            // decision 2
        if (balance >= amount) { // decision 3
            processPayment();
        }
    }
}
return;
```

The tester drew a straight chain 1-2-3-4-5-9 and omitted every false edge. One test (valid user, positive amount, enough balance) executes every drawn statement, giving **100% statement coverage**, while none of the three false (error) branches is run: **0 of 3 error branches, 3 of 6 branches overall (50%)**.

#	Graph construction error	Consequence
E1	False edge of 'userValid' missing	Invalid user handling untested.
E2	False edge of 'amount > 0' missing	Zero or negative amount reaches payment unchecked.
E3	False edge of 'balance >= amount' missing	Overdraft defect remains hidden.
E4	Implicit else not represented as a node	Join node 9 receives only one incoming edge.
E5	V(G) computed as 1	Real complexity 4 was underestimated; only 1 of 4 paths tested.

Table 1: Errors in graph construction

2. Root Cause Analysis

- **Statement-driven thinking:** nodes were created only for lines of code, and an absent `else` has no line, so no node appeared.
- **Decision nodes treated as statements:** each `if` must have two outgoing edges (True, False) regardless of whether an else block is written.
- **Nested structure flattened:** the nesting was read as a sequence, so each false outcome that jumps straight to the join node was lost.
- **No cross-check with V(G):** predicates + 1 = 4 would have shown the drawn graph (V = 1) was wrong.

3. Corrected CFG and Independent Paths

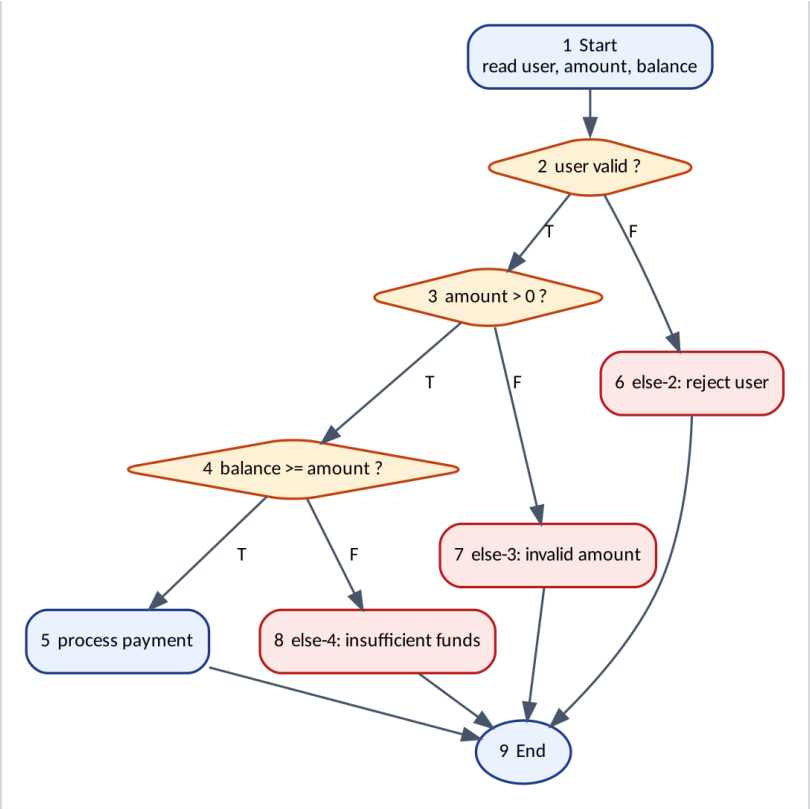


Figure 1: Corrected CFG with every else branch represented

Cyclomatic complexity: N = 9, E = 11, so **V(G) = 11 - 9 + 2 = 4**. Predicate count: 3 decisions + 1 = 4. Four independent paths are therefore required.

Path	Node sequence	Test input	Expected result
P1	1-2-3-4-5-9	valid user, amount 500, balance 1000	Payment processed
P2	1-2-3-4-8-9	valid user, amount 500, balance 200	Insufficient funds
P3	1-2-3-7-9	valid user, amount 0 (or -50)	Invalid amount
P4	1-2-6-9	invalid user	Access rejected

Table 2: Independent paths

Branch (edge)	Covered by	Before (1 test)	After (4 tests)
2 to 3 (T)	P1, P2, P3	Yes	Yes
2 to 6 (F)	P4	No	Yes
3 to 4 (T)	P1, P2	Yes	Yes
3 to 7 (F)	P3	No	Yes
4 to 5 (T)	P1	Yes	Yes
4 to 8 (F)	P2	No	Yes
Branch coverage		**3/6 = 50%**	**6/6 = 100%**

Table 3: Branch coverage improvement

4. Branch Adequacy Justification

- Each decision outcome is now an edge in the graph and is traversed by at least one test, achieving 100% branch coverage.
- The four paths form a **basis set**: any other path is a combination of them, so branch adequacy is reached with the minimum number of tests.
- Error paths (P2, P3, P4) are precisely where production incidents arise (rejected users, overdrafts), so covering them has the highest risk reduction.

5. Conclusion

A CFG must show both outcomes of every decision, even when the else block is empty. The corrected graph (V(G) = 4) and its four independent paths turn a misleading 100% statement coverage into true branch adequacy.